```
a[1] = 'bat'              #=> [1, "bat", 5, 7, 9]
a[-3] = 'cat'             #=> [1, "bat", "cat", 7, 9]
a[3] = [9, 8]             #=> [1, "bat", "cat", [9, 8], 9]
a[6] = 99                 #=> [1, "bat", "cat", [9, 8], 9, nil, 99]
```

Again, []= is just a regular method, and you could write it as a.[]=(index, new_value).

If the index to []= is two numbers (a start and a length) or a range, then those elements in the original array are replaced by whatever is on the right side of the assignment. If the length of the selected elements on the left is zero, the right side is inserted into the array before the start position; no elements are removed. If the right side is itself an array, its elements are used in the replacement. The array size is automatically adjusted if the index selects a different number of elements than are available on the right side of the assignment.

```
a = [1, 3, 5, 7, 9]       #=> [1, 3, 5, 7, 9]
a[2, 2] = "cat"           #=> [1, 3, "cat", 9]
a[2, 0] = "dog"           #=> [1, 3, "dog", "cat", 9]
a[1, 1] = [9, 8, 7]       #=> [1, 9, 8, 7, "dog", "cat", 9]
a[0..3] = []              #=> ["dog", "cat", 9]
a[5..6] = 99, 98          #=> ["dog", "cat", 9, nil, nil, 99, 98]
```

In the line a[2, 2] = "cat", the subarray starting at index 2 and of length 2, which is [5, 7] is replaced by cat. In the next line, the subarray [2, 0] is of length 0, so dog is just inserted at index 2. Then the subarray represented by [1, 1], which is [3] is replaced by [9, 8, 7] being inserted in the array—notice that the entire right-side array is not inserted as one element, rather each element in the right hand side is inserted individually. The last two lines are similar, but use ranges instead of start and length.

It's common to create arrays of short words, but that can be a pain, what with all the quotes and commas. Fortunately, Ruby has a shortcut; %w does just what we want:

Instead of this:

```
a = ["ant", "bee", "cat", "dog", "elk"]
a[0]      # => "ant"
a[3]      # => "dog"
```

You can use %w followed by a delimiter, followed by space-separated individual words.

```
a = %w[ant bee cat dog elk]
a[0]      # => "ant"
a[3]      # => "dog"
```

You can use any character after %w as the delimiter, if it's something with a pair, like a bracket or a parenthesis, then the array will continue until the other side of the pair, otherwise the array will continue until it reaches the same character again.

If you want an array of symbols instead of strings, Ruby has a similar %i delimiter:

```
a = %i[ant bee cat dog elk]
a[0]      # => :ant
a[3]      # => :dog
```

Arrays have a large number of other useful methods. Using them, you can treat arrays as stacks, sets, queues, dequeues, and first-in first-out (FIFO) queues. (Though Ruby does also